

HW/Lab 2: SNR, BER, Sampling, Hardware

ECSE 4560/6560: Modern Communication Systems

RPI Fall 2025

Goals and Instructions

The goals for this assignment are:

- Implement a Monte Carlo BER simulation for coherent BPSK in AWGN and compare it against the analytical curve.
- Build a Simulink + Pluto(+) SDR OOK chain to empirically measure and plot BER vs SNR.

Follow these instructions:

- **Collaboration:** You can discuss concepts, but write your own solutions and code.
- **Tools:** We will primarily use MATLAB (and/or Python). Directions use standard MATLAB functions. *Strongly recommended* toolboxes: Signal Processing, Communications. (Base-MATLAB fallbacks are acceptable.)
- **Hardware:** We use PlutoSDR or Pluto+ SDR for this exercise.
- **Submission:** upload a single PDF with your solutions and MATLAB code to Gradescope before the deadline. You may use MATLAB's **Publish** feature to generate a compiled PDF and attach it as your single submission. Include your name on the PDF.

Matlab Simulations: BPSK BER Simulation in AWGN

Instructions: You will implement a Monte Carlo BER simulation for coherent BPSK in AWGN, compare against the analytical BER curve, and then study the *modeled* effect of oversampling by scaling the per-sample noise variance as $1/L$. A MATLAB template with `TODO` placeholders is provided on Piazza. Use `rng(123)` for reproducibility. You may submit either (a) your `.m` content plus a short PDF with the figures and answers in a single pdf, or (b) a single published MATLAB report. For detailed theory on this exercise, follow the pdf posted on Piazza resources called "[Bit-Error-Rate Simulation Using Matlab](#)".

1. BER vs. SNR for BPSK (match theory).

- a) **Parameters:** Set the number of bits to simulate to `num_bits = 1e5`, and the SNR grid to `snr_db_vec = 0:10`. Initialize the random generator with `rng(123)`. Preallocate arrays as needed.

- b) **Symbol generation:** Generate i.i.d. bits `bits` in $\{0,1\}$ and map to BPSK symbols $\{-1,+1\}$ using `bpsk_syms = 2*bits - 1`.
 - c) **Noise scaling:** For each SNR point in dB, compute the linear ratio $\text{ebn0} = 10^{(\text{snr_db}/10)}$. Use the normalization $E_b = 1$. Compute $N_0 = E_b/\text{ebn0}$.
 - d) **Detection and BER:** Perform hard decisions with threshold 0 (i.e., $\hat{x} = +1$ if $y \geq 0$, else -1). Compute the empirical BER by counting symbol errors.
 - e) **Analytical curve:** Compute and plot the theoretical BER $P_b = Q(\sqrt{2E_b/N_0}) = 0.5\text{erfc}(\sqrt{E_b/N_0})$ on the same axes (semilog-y). Label axes, add legend, and ensure grid is on. This plot is Figure 1.
 - f) **Brief reflection (3–5 sentences):** Comment on the similarity between the simulated and analytical curves, the effect of Monte Carlo variability at low/high SNRs, and any minor discrepancies you observe.
2. (ECSE 6560 only) **Modeled oversampling study (processing-gain view).**
- a) **Oversampling factors:** Let `oversamp_fac_vec = [1, 2, 4]`. For each L in this set, *scale the per-sample noise variance by $1/L$ (i.e., use `N0_effective = N0 / L` before forming the variance `N0_effective/2`). No matched filtering;* keep the detector and symbol rate identical to Question 1.
 - b) **BER computation:** For each L and each SNR point, generate noise with variance $(N_0/L)/2$, add to the same `bpsk_syms`, hard-detect, and record BER. Preallocate a matrix of size `[numel(oversamp_fac_vec), num_snrs]` for efficiency.
 - c) **Plot:** Plot three BER curves (one per L) on a new figure (Figure 2), semilog-y, with legend, labels, title, and grid.
 - d) **Short discussion (bullet points or 3–5 sentences):**
 - Explain why the BER appears to improve as L increases under this *noise-scaling* model.

Hardware Experiment

In this part, we use a PLUTO+ SDR to simulate OOK transmission, which sends a cosine wave for bit 1, and sends nothing for bit 0 (noise only). We will install the software for the SDR, build a simulation model in MATLAB Simulink, and run the simulation to plot SNR versus BER.

1. Installation

- a) Connect the SDR's DATA interface to the computer's USB port. You can leave the 5V-IN port disconnected as long as the USB port supplies power. Make sure the SDR's blue LED is on, while its green LED blinks slowly. There should be a USB drive named PlutoSDR shown in the file explorer.
- b) (Only for new PLUTO+ SDR) Download the [PLUTO firmware](#) to the PlutoSDR USB drive (don't unzip it). Right-click on PlutoSDR in the file explorer to eject it (don't use the taskbar to eject). Keep the SDR plugged in, and wait until it reconnects automatically.

- c) Open MATLAB (version 2025b highly recommended, other versions may have different configurations) and go to HOME -> Add-Ons. Install the following Add-Ons with the default suggested installation settings.
 - i. Simulink
 - ii. Signal Processing Toolbox
 - iii. Communications Toolbox
 - iv. DSP System Toolbox
 - v. Communications Toolbox Support Package for Analog Devices ADALM-Pluto Radio

In the ‘set up’ of the Pluto Radio Add-On, MATLAB will also update the firmware. After ejection, if the SDR does not reconnect automatically, make sure the LEDs are not blinking fast, then unplug and replug in the SDR to connect.

- d) Check the connection of the SDR by the MATLAB command **findPlutoRadio**. Make sure it prints out the radio ID and serial number, as shown in Fig. 1.



```

Command Window
>> findPlutoRadio

ans =

struct with fields:

    RadioID: 'usb:0'
    SerialNum: '104000d60d13000621002b0023416f6308'

>> |
  
```

Figure 1: **findPlutoRadio** Output.

- e) If, in a second-time use on Windows, the PLUTO+ SDR has its LEDs on but does not appear in the file explorer, try the following steps:
 - i. Go to Control Panel -> Device Manager -> view -> show hidden devices
 - ii. Expand Universal Serial But Controllers
 - iii. Uninstall USB Mass Storage Device
 - iv. Replug in the SDR

Deliverable: Include a screenshot of the **findPlutoRadio** output.

2. **Simulink Model** We simulate OOK with $\{0, 1\}$ binary symbols (bits), using cosine (real) modulation. Under OOK, for bit 1, we send a cosine wave with a positive amplitude, while for bit 0, we send a cosine wave with zero amplitude. In the simulation, we send 1×10^5 symbols (bits) per second, and for both DAC and ADC, we use a sample rate of 1×10^6 Hz, which means we always take 10 samples per symbol. We use a 1×10^5 Hz sine wave for pulse shaping (this is also an upconversion). Then, we modulate the shaped pulses onto a carrier cosine wave with frequency 9.15×10^8 Hz.

For the simulation, we set one frame per second, and we will run the simulation for several frames. We will let the transmitter send only bit 0s at the beginning, then send only 1s in the rest of the frames. As a result, the receiver will receive only the noise power at the beginning, then receive both the signal and noise power.

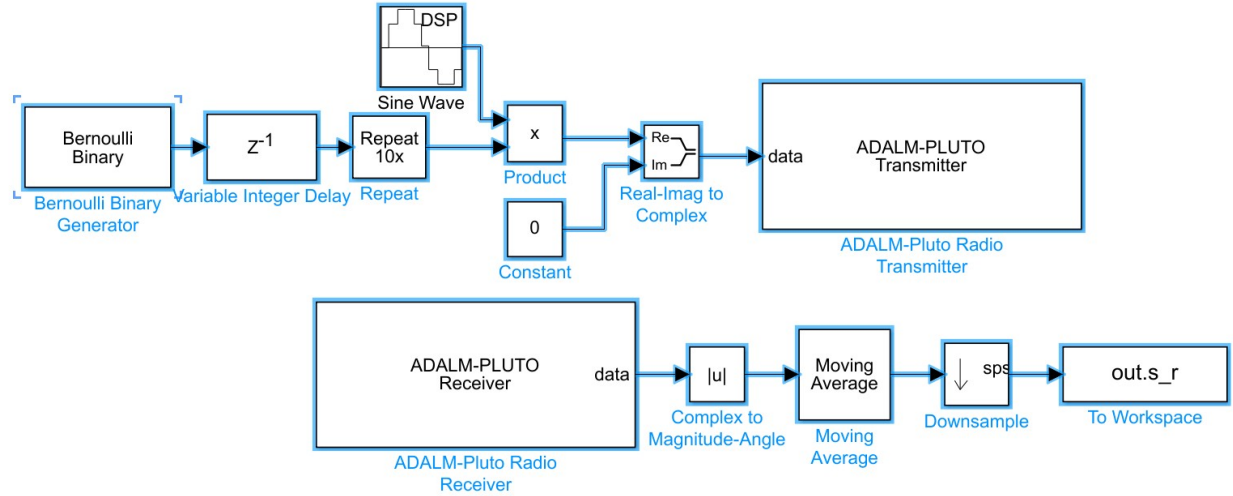


Figure 2: Simulink Model.

- a) Run the following code to initialize the parameters. Please note that the Simulink model will share the same workspace as the MATLAB code.


```
fs = 1e6; % sample rate
sps = 10; % samples per symbol
fc = 915e6; % carrier frequency (inside unlicensed band)
n = 1e5; % number of bits or symbols per frame (1 bit per symbol)
m = 5; % Simulink will run (m+1) frames
G = 65; % input power gain in dB
A = -70; % transmitter attenuation in dB
D = 2; % input delay
```
- b) Open Simulink by the MATLAB command **simulink**. Create a Blank Model, and save it as 'lab2.slx'.
- c) In the Simulink window, go to SIMULATION -> Library Browser. You can search the blocks here, and drag them out to insert into our model.
- d) Build the model as shown in Fig. 2. The configuration settings for each block are as follows. Leave the unmentioned settings as the default. There are blocks from different toolboxes with identical names but slightly different functionalities. Please choose the ones as mentioned.
 - i. **Bernoulli Binary Generator** (Communications Toolbox - 1)
This block generates bits. Here, we let it only generate 1s.
 Probability of zero: 0, Sample time: sps/fs, Sample per frame: n
 - ii. **Variable Integer Delay** (Simulink)
We delay the input for D frames, so in the first D frames we will send only 0s.
 Delay length: Diagonal D
 - iii. **Repeat** (DSP System Toolbox - 2)
Take sps samples per bit.
 Repetition count: sps

- iv. **Sine Wave** (DSP System Toolbox - 1)
This block generates a sampled sine wave, which we use for pulse shaping.
 Frequency (Hz): n , Sample time: $1/f_s$, Samples per frame: $n \cdot \text{sps}$
 - v. **Product** (Simulink)
 Default
 - vi. **Constant** (Simulink)
We don't send anything in the imaginary part.
 Constant value: 0
 - vii. **Real-Imag to Complex** (Simulink)
The Pluto TX block requires a complex input.
 Default
 - viii. **ADALM-Pluto Radio Transmitter**
This block is connected to the Pluto+ SDR, which will execute the upconversion and send the signal.
 Center frequency (Hz): f_c , Gain (dB): $A+G$, Baseband sample rate (Hz): f_s , Uncheck output port for underflow
 - ix. **ADALM-Pluto Radio Receiver**
This block is connected to the Pluto+ SDR, which will receive the signal and execute the downconversion.
 Center frequency (Hz): f_c , Source of gain: Dialog, Baseband sample rate (Hz): f_s , Output data type: double, Samples per frame: $n \cdot \text{sps}$, Uncheck output port for overflow
 - x. **Complex to Magnitude-Angle** (Simulink)
Because there is a channel phase shift, the received signal is no longer entirely in the real part. The received magnitude should be obtained from both real and imaginary parts.
 Output: Magnitude
 - xi. **Moving Average** (DSP System Toolbox - 1)
This block makes each sample equal to the mean of itself and its previous ($\text{sps} - 1$) samples, which reverses our sine-pulse shaping.
 Window length: sps , Overlap length: $\text{sps}-1$
 - xii. **Downsample** (DSP System Toolbox - 3)
This block picks one sample for each symbol.
 Downsample factor, M : sps
 - xiii. **To Workspace** (Simulink)
Output the received magnitudes to the workspace.
 Variable name: `s_r`
- e) The Time Scope (DSP System Toolbox - 1) can be used to observe the signal in the time domain. To verify that the model works, additionally connect the Pluto RX block output to a Time Scope. Then, go to Simulation -> Stop Time and set the simulation time to 5. Press Run to start the simulation. Simulink will run for $(5 + 1)$ seconds, which is exactly 6 frames in our case. The Time Scope window will automatically pop up. It will update the waveform, frame by frame. At last, you should see something as shown in Fig. 3. There are some impulses caused by the startup of the SDR at the beginning. After that, there should be an only-noise period with magnitude close to 0. At last,

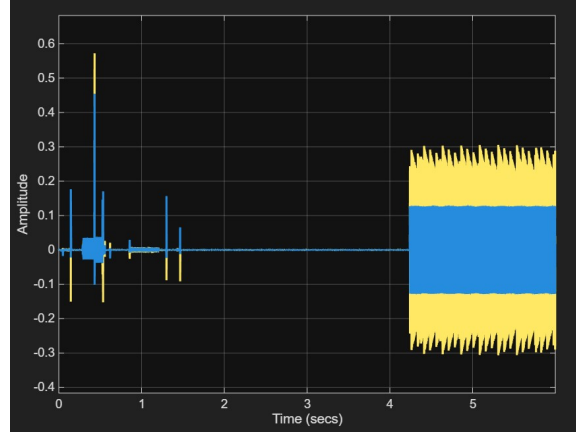


Figure 3: Time Scope Window.

there should appear a strong blue wave (real part) and a strong yellow wave (imaginary part), which are the bit 1 stream we sent. Since the channel phase shift varies, the yellow wave might be covered by the blue wave (or vice versa), which is acceptable.

- f) We need at least one frame that is purely ‘only-noise’, i.e., does not contain any impulses or signal waves. We also need the last frame to be filled with the signal wave, which means the signal wave should appear before the last frame. Please satisfy these two requirements by adjusting the Stop Time in the Simulink window, and the input delay D in the workspace (run ‘ $D=...$ ’ in the command window).
- g) Once you determine the Stop Time and the input delay, update the values to the initialization code for m and D . In the Time Scope window, go to SCOPE -> Print to Figure, and save as a JPG file.

Deliverable: Include a screenshot of the Simulink model and a figure print from the Time Scope window.

3. Plot SNR vs BER

- a) Initialize all parameters with the updated m and D values. We can then use code to run the Simulink model. We first load the model with the code:


```
md = 'lab2.slx';
load_system(md);
```
- b) Then, we can run the simulation for $(m + 1)$ frames by:


```
out = sim(md, 'StopTime', num2str(n*sps/fs*m));
```

 Here, m specifies m frames, and Simulink will add a frame.
- c) Now we need to obtain the array of magnitude per symbol (also per bit) in the only-noise frame. Suppose the third frame is the only-noise frame, the array of the received magnitude can be obtained by:


```
mag_noise=out.s_r.Data{3};
```

 Note that we can obtain noise power P_N by taking the mean of the squared magnitudes.
- d) Similarly, we can obtain the magnitude per symbol in the last frame (the frame filled with signal wave) by:


```
mag_total=out.s_r.Data{end};
```

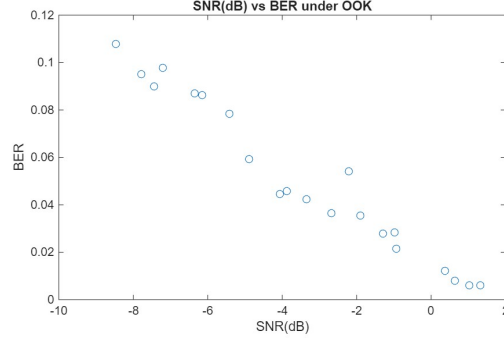


Figure 4: BER Plot.

In the last frame, the receiver receives both the signal and the noise, so the mean of squared magnitudes is the sum of signal power and noise power. We can estimate the signal power P_S by subtracting the noise power P_N from the total power. Please note that we set the Bernoulli Binary block to always output 1, so the signal power P_S is twice the empirical signal power.

- e) We can calculate the SNR(dB) from P_S and P_N . Please use the empirical signal power for the SNR. Because we measure the noise power in a different frame, there exists a possibility that the measured P_N is larger than the total power in the last frame. In this case, check if the frame of P_N is only-noise, if so, just rerun the simulation.
- f) We set the decision boundary as half of the **mag_total** mean. The BER can be calculated as the probability of reconstructing bit 0 from **mag_total**.
- g) Traverse gain **G** in the range 0:0.5:10, and plot SNR(dB) vs BER. Adjust the transmitter attenuation A , so the lowest SNR plotted is around -10 dB. Please note:
 - i. In each iteration, you need to set the value of **G**, run the simulation, and calculate SNR and BER.
 - ii. If you happen to get a negative P_S , try to rerun the simulation. If it happens frequently, set a larger A to increase SNR.
 - iii. Instead of a line plot, please use a scatter plot, e.g., MATLAB code:
`plot(SNR_set,BER_set,'o')`
 - iv. You should obtain something similar to Fig. 4.

Deliverable: Include the SNR(dB) vs BER plot.

Note: Figures must include titles, labeled axes with units, and legends when relevant.